# Line-Following Robot: Software Architecture and API Documentation

Eric

April 3, 2026

## Contents


**1 System Overview** **3**

**2 Hardware Abstraction Layer API** **3**
- 2.1 `MotorDriveBoard` . . . 3
- 2.2 `SensorArray` . . . 4

**3 Core Data Structures** **4**
- 3.1 `LineInfo` . . . 4
- 3.2 `WheelPIState` . . . 5
- 3.3 `DebugSample` and `CompactDebugSample` . . . 6

**4 Track Feature Extraction and Visibility Detection** **6**
- 4.1 Dark-Line / Bright-Line Model . . . 6
- 4.2 Lateral Position Estimation . . . 6
- 4.3 Visibility Decision Logic . . . 7

**5 Control Loops and Strategy** **7**
- 5.1 Outer Loop: Dynamic PD + Curve-Score Scheduling . . . 7
- 5.2 Curve Score . . . 7
- 5.3 Speed Scheduling and Straight-Line Boost . . . 8
- 5.4 Steering Saturation Warning and Preventive Slowdown . . . 9
- 5.5 Final Outer-Loop Output . . . 9

**6 Inner Loop: Wheel-Speed PI + Feedforward** **9**
- 6.1 Encoder-Based Speed Estimation . . . 9
- 6.2 PI + Feedforward Control Law . . . 10
- 6.3 Safety and Output Conditioning . . . 10

**7 State Machine (`CarState`)** **10**
- 7.1 `STATE_TRACKING` . . . 10
- 7.2 `STATE_LOST_SEARCH` . . . 11
- 7.3 `STATE_STOPPED_RECOVERY` . . . 11
- 7.4 `STATE_TURNAROUND_180` . . . 11

**8 Run Arming and Recovery Strategy** **12**

**9 Diagnostics and Dual-Level Zero-Blocking Logging** **12**
9.1 Motivation . . . 12
9.2 Short-Term Detailed Buffer . . . 13
9.3 Long-Term Compressed History . . . 13
9.4 Freeze and Dump Mechanism . . . 14

**10 Key Parameter Summary (Current Tuned Parameters)** **14**
10.1 Control Period and Sensors . . . 14
10.2 Speed Strategy . . . 14
10.3 Outer Loop . . . 15
10.4 Inner Loop . . . 15
10.5 State Machine . . . 15
10.6 Turnaround Parameters . . . 15

**11 Conclusion** **16**

# 1 System Overview

This document describes the software architecture of a line-following robot based on **mbed OS 2**. The system runs at a fixed control rate of **200Hz (5ms)** and adopts a **cascade control architecture**:

- **Perception layer**: the infrared sensor array performs ambient-light compensation and extracts track features;
- **Outer loop**: a **dynamic PD + speed scheduling** controller based on position error and error rate;
- **Inner loop**: a **wheel-speed PI + feedforward** controller based on encoder feedback;
- **Strategy layer**: a state machine handles arming, line-loss protection, automatic recovery, and BLE-triggered turnaround;
- **Diagnostics layer**: a dual-level zero-blocking ring-buffer logger (short detailed history + long compressed history).

The system does not start driving immediately after power-up. Instead, it first enters an **unarmed waiting state**. Only after the sensors detect the track continuously for a certain duration is the robot allowed to enter normal tracking mode. This design significantly reduces false starts and unintended motion during initialization.

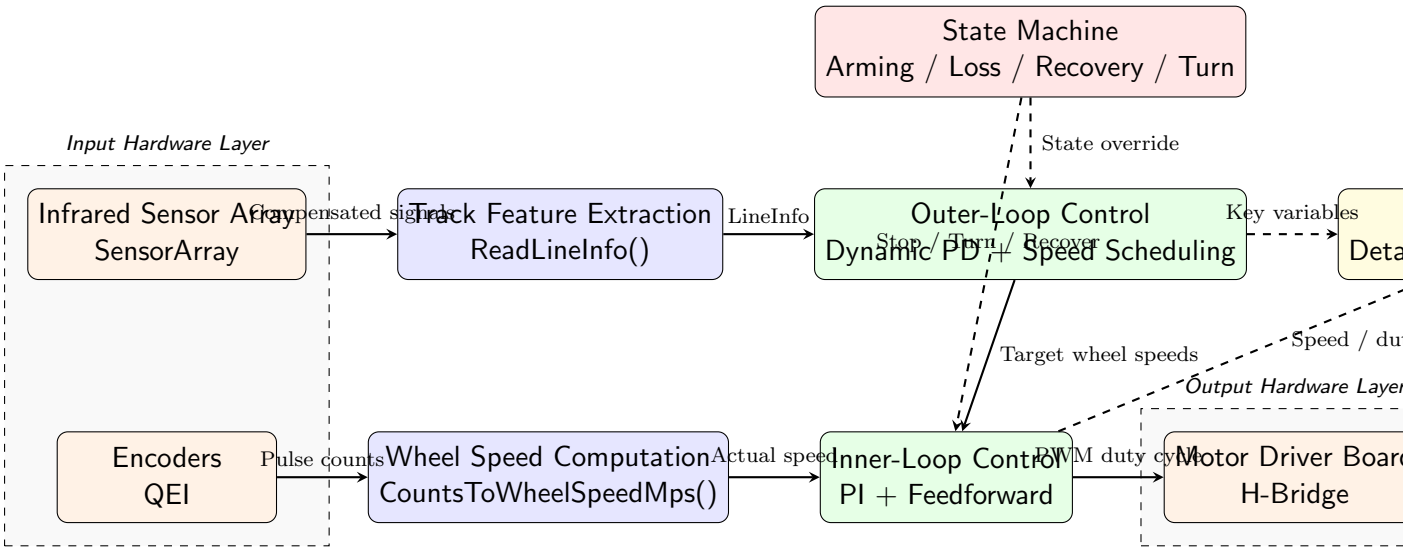


Figure 1: Data flow and module architecture of the line-following robot

# 2 Hardware Abstraction Layer API

## 2.1 `MotorDriveBoard`

`MotorDriveBoard` encapsulates the left and right motor PWM outputs, direction pins, mode pins, and enable signal. Its role is to provide a clean software abstraction for the underlying H-bridge motor driver.

- `void SetDutyCycle(float l, float r)` Clamps the left and right duty cycles to the range `[0.0, 1.0]` and automatically handles the required **inverted PWM** logic at the hardware level:

$$PWM_{\text{out}} = 1.0 - duty$$

- `void SetPeriod(float l, float r)` Sets the PWM period for the left and right channels. In the current implementation, the period is initialized to `0.001s` (approximately 1kHz).
- `void SetDirection(int l, int r)` Sets the motion direction of the left and right wheels.
- `void SetMode(int l, int r)` Sets the mode pins of the motor driver.
- `void SetEnable(int e)` Enables or disables the motor driver board.

### 2.2 SensorArray

`SensorArray` manages six infrared sensors and their corresponding emitter LEDs. The system uses an **active ambient-light cancellation** strategy to reduce environmental interference.

- `void SetAllLEDs(int on)` Turns all infrared emitter LEDs on or off simultaneously.
- `void ReadCompensated(float signal[6])` Performs one compensated sampling cycle:
  1. Turn LEDs off and wait for `LED_SETTLE_US = 20` $\mu$s;
  2. Read six ambient-light ADC values;
  3. Turn LEDs on and wait another `20` $\mu$s;
  4. Read six "ambient + reflected light" ADC values;
  5. Subtract the two measurements to obtain the net reflected signal.

  Negative differences are clamped to zero. This method suppresses ambient brightness variation, surface reflections, and external light interference.

## 3 Core Data Structures

### 3.1 LineInfo

`ReadLineInfo()` converts the compensated six-channel sensor readings into higher-level track features for the controller. In the current implementation, `lineOffset` is **not** simply normalized to `[-1,1]`. Instead, it is a **weighted lateral position value** computed from the sensor-weight array, with an approximate range determined by those weights (about `-450` to `450` in this implementation).

```
struct LineInfo {
    bool lineVisible;
    float lineOffset;

    float minSig;
```

```
    float maxSig;
    float contrast;

    float rawSig[6];
    float normSig[6];

    float lineStrength;
    float pairMax;

    float leftSum;
    float rightSum;
    float lrBias;
};
```

The fields are interpreted as follows:

- `lineVisible`: whether the system considers the line reliably visible in the current cycle;
- `lineOffset`: lateral offset of the track center relative to the vehicle;
- `minSig / maxSig`: minimum and maximum raw compensated signal values;
- `contrast`: `maxSig - minSig`, representing local contrast;
- `rawSig[6]`: six compensated raw reflection values;
- `normSig[6]`: effective normalized signals after dark-line / bright-line processing;
- `lineStrength`: sum of effective signal strengths;
- `pairMax`: maximum adjacent-pair response, used to reject isolated noise;
- `leftSum / rightSum`: summed responses of the left and right sensor groups;
- `lrBias`: left-right energy imbalance, useful for identifying turning tendency.

## 3.2 `WheelPIState`

The persistent internal state of the wheel-speed controller is stored in `WheelPIState`:

```
struct WheelPIState {
    float targetL, targetR;
    float dutyL, dutyR;
    float integralL, integralR;
    float dutyMin, dutyMax;
    float integralLimit;
    float maxDutyStep;
    float ffGain;
};
```

This structure stores:

- left and right target wheel speeds;
- current PWM duty cycles for both wheels;
- PI integrator states for both wheels;

- duty-cycle upper and lower bounds;
- integrator limit;
- maximum duty-cycle change per control step (slew-rate limiter);
- feedforward gain.

### 3.3 `DebugSample` and `CompactDebugSample`

The system maintains two logging formats:

- `DebugSample`: short-term detailed logs containing raw sensor data and full control variables;
- `CompactDebugSample`: long-term compressed logs using scaled fixed-point fields for extended history.

This layered design provides both **high-resolution crash reconstruction** and **long-duration trend retention**.

## 4 Track Feature Extraction and Visibility Detection

### 4.1 Dark-Line / Bright-Line Model

The current code uses:

`TRACK_IS_DARK_LINE = true`

which means the system assumes a **dark-line-on-brighter-background** model. Therefore, the effective line signal is computed as:

$$val_i = maxSig - rawSig_i$$

If a bright-line model is used instead, the expression becomes:

$$val_i = rawSig_i - minSig$$

To suppress small fluctuations and weak noise, only values above a fraction of the total contrast are retained:

$$val_i = 0 \quad \text{if} \quad val_i < 0.20 \cdot contrast$$

### 4.2 Lateral Position Estimation

The system uses the following sensor position weights:

$$[-450,\ -150,\ -50,\ 50,\ 150,\ 450]$$

Thus, the lateral position is computed as:

$$lineOffset = \frac{\sum_i val_i \cdot w_i}{\sum_i val_i}$$

If the total effective signal is too small, the offset is set to zero.

### 4.3 Visibility Decision Logic

The current implementation does not rely on a single threshold. Instead, it checks several conditions simultaneously:

- `contrast >= LINE_CONTRAST_MIN`
- `maxSig >= LINE_RAW_MAX_MIN`
- `lineStrength >= LINE_STRENGTH_MIN`
- `pairMax >= LINE_PAIR_MIN`

Only when all these conditions are satisfied is `lineVisible` set to true. This multi-condition strategy is more robust than a single-sum threshold and is less sensitive to background reflections, local noise, and isolated bright or dark artifacts.

## 5 Control Loops and Strategy

### 5.1 Outer Loop: Dynamic PD + Curve-Score Scheduling

The outer loop input is:

$$error = -lineOffset$$

and the derivative term is computed from the change in error. To reduce jitter, the derivative first passes through a first-order low-pass filter and is then limited:

$$D_{filt}(k) = \alpha D_{raw}(k) + (1 - \alpha) D_{filt}(k - 1)$$

where:

$$\alpha = \texttt{D_FILTER_ALPHA}$$

If the error falls within the center deadband `CENTER_DEADBAND`, the turning error is treated as zero to avoid unnecessary correction due to very small noise near the center.

### 5.2 Curve Score

The system does not treat any single variable as the complete turning indicator. Instead, it combines three features:

- **Position error**: how far the robot is from the line center;
- **Error rate**: how fast the deviation is changing;
- **Left-right bias**: how imbalanced the left and right sensor energies are.

The code first normalizes these terms:

$$e_{norm} = clamp\left(\frac{|error|}{120}, 0, 1\right)$$

$$d_{norm} = clamp\left(\frac{|derivative|}{600}, 0, 1\right)$$

$$c_{norm} = clamp(lrBias, 0, 1)$$

Then it forms the raw curve score:

$$curveRaw = 0.50\, e_{norm} + 0.15\, d_{norm} + 0.35\, c_{norm}$$

This value is filtered to obtain a smooth curve score:

$$curveScore(k) = 0.10 \cdot curveRaw + 0.90 \cdot curveScore(k-1)$$

The curve score is used to dynamically schedule:

- **Proportional gain** `Kp`;
- **Derivative gain** `Kd`;
- **Maximum steering authority** `maxSteer`;
- **Base forward speed** `baseSpeed`.

## 5.3 Speed Scheduling and Straight-Line Boost

The base speed is not fixed. Instead, it changes continuously with curve severity:

$$desiredBase = SPEED_CRUISE - (SPEED_CRUISE - SPEED_TIGHT)\sqrt{curveScore}$$

This means:

- on a straight section, the base speed approaches `SPEED_CRUISE`;
- in a sharp corner, the base speed quickly drops toward `SPEED_TIGHT`;
- the square-root shape causes stronger early deceleration in medium-to-high curvature conditions.

The system also includes a **straight-line boost confirmation mechanism**. The robot enters high-speed straight mode only if all of the following remain true for a sufficient duration:

- the line is visible;
- the absolute error is small;
- the absolute derivative is small;
- `lrBias` is small;
- the curve score is low;
- these conditions persist longer than `BOOST_ENTER_TIME_S = 0.1s`.

When that happens, the base speed is elevated to:

$$baseSpeed = SPEED_STRAIGHT$$

## 5.4 Steering Saturation Warning and Preventive Slowdown

To reduce the risk of overshooting a corner at high speed, the controller includes a **preemptive slowdown mechanism** when the estimated steering demand approaches saturation.

If the predicted steering satisfies:

$$|steer| > STEER_SAT_RATIO \cdot maxSteer$$

for a short duration, the controller will:

- force the base speed down to `STEER_SAT_REDUCED_SPEED`;
- immediately cancel the current straight-line boost;
- reset the boost confirmation timer.

This acts as a preventive braking strategy and helps reduce understeer and corner-exit overshoot.

## 5.5 Final Outer-Loop Output

The outer loop generates the left and right target wheel speeds as:

$$target_L = baseSpeed - steerSpeed$$

$$target_R = baseSpeed + steerSpeed$$

The targets are then clamped to:

$$target \in [TARGET_SPEED_MIN_TRACK,\ TARGET_SPEED_MAX_TRACK]$$

# 6 Inner Loop: Wheel-Speed PI + Feedforward

## 6.1 Encoder-Based Speed Estimation

The pulse increments from the encoders are converted to wheel speed using:

$$v = \left( \frac{\Delta counts}{COUNTS_PER_MOTOR_REV \cdot GEAR_RATIO \cdot dt} \right) \cdot 2\pi R$$

Because the current hardware has encoder polarity opposite to the software-defined forward direction, the code applies a sign correction:

$$speed_L = -CountsToWheelSpeedMps(\Delta L, dt)$$

$$speed_R = -CountsToWheelSpeedMps(\Delta R, dt)$$

## 6.2 PI + Feedforward Control Law

The inner-loop control command consists of three parts:

$$cmd = ffGain \cdot target + K_p \cdot error + K_i \cdot integral$$

where:

- the **feedforward term** provides a duty-cycle baseline from the target speed;
- the **proportional term** corrects speed error quickly;
- the **integral term** removes steady-state error.

## 6.3 Safety and Output Conditioning

The inner loop also includes several protection mechanisms:

- **integrator limiting**: prevents unlimited accumulation of the integral term;
- **duty clamping**: keeps the output within `[DUTY_MIN, DUTY_MAX]`;
- **slew-rate limiting**: limits duty change per 5ms cycle to `DUTY_STEP_LIMIT`;
- **hard-stop reset**: clears targets and integrators during loss-stop and recovery transitions.

This reduces wheel slip, current spikes, and abrupt motion caused by overly aggressive duty changes.

# 7 State Machine (`CarState`)

The current system defines four operating states:

```
enum CarState {
    STATE_TRACKING = 0,
    STATE_LOST_SEARCH = 1,
    STATE_STOPPED_RECOVERY = 2,
    STATE_TURNAROUND_180 = 3
};
```

## 7.1 `STATE_TRACKING`

This is the normal line-following mode. In this state, the system runs:

- track perception;
- outer-loop dynamic PD control;
- inner-loop wheel-speed PI control;
- continuous background logging.

## 7.2 STATE_LOST_SEARCH

Although the name LOST_SEARCH is kept, the current implementation does **not** actively perform a search maneuver. Its real role is:

- it is triggered when the system first loses the line during normal tracking;
- it immediately freezes the current log buffers;
- it immediately stops motor output;
- it provides a very short transition window during which the line may be reacquired quickly.

If the line remains lost for longer than

$$LOST_STOP_TIME_S = 0.025s$$

the system transitions to STATE_STOPPED_RECOVERY.

## 7.3 STATE_STOPPED_RECOVERY

This is the **safe stopped waiting-for-recovery state**. Its behavior is:

- motor output is forcibly stopped;
- PI state is cleared;
- if the line becomes visible again, a recovery timer starts accumulating;
- the system returns to TRACKING only after the line remains stably visible for

$$RECOVERY_GO_DELAY_S = 0.20s$$

This prevents the robot from restarting abruptly when it has only just been placed back onto the line and the sensor signal is still unstable.

## 7.4 STATE_TURNAROUND_180

When the BLE serial input receives the character 'A' or 'a', and the system has already been armed, the robot enters the 180-degree turnaround state. Its characteristics are:

- encoder counts are used to measure left and right wheel rotation;
- TURN_180_TICKS is used as the target turn amount;
- a mainly open-loop base duty is applied;
- a small synchronization correction term TURN_SYNC_KP is added to reduce accumulated wheel mismatch;
- when the remaining turn enters the slow zone, the duty is automatically reduced.

After the turn finishes:

- if the line is already visible, the system returns directly to TRACKING;
- otherwise it transitions into STATE_LOST_SEARCH and waits for later recovery.

# 8 Run Arming and Recovery Strategy

The system does not enter tracking mode immediately after power-up. Instead, it requires the line to remain continuously visible for:

$$RUN_ARM_VISIBLE_S = 0.25s$$

before setting `g_runArmed` to true and allowing:

- normal closed-loop tracking;
- BLE-triggered turnaround commands;
- the full state machine logic to operate.

This mechanism helps prevent:

- accidental startup when the robot is not placed on the track;
- false launch caused by transient sensor signals during initialization;
- turnaround commands being accepted before the system is ready.

In addition, once the line has been stably visible again for more than

$$FROZEN_CLEAR_VISIBLE_S = 0.15s$$

the system clears previously frozen log snapshots so that old incident records do not interfere with later diagnosis.

# 9 Diagnostics and Dual-Level Zero-Blocking Logging

## 9.1 Motivation

If `printf()` is called continuously inside a 5ms control loop, serial I/O can significantly block execution and cause:

- control-period jitter;
- degraded real-time behavior;
- distorted controller response;
- stuttering or even unstable vehicle motion.

Therefore, the system uses a **record-first, dump-later** black-box logging strategy: during runtime, data is silently stored in RAM; when an incident occurs, the history is dumped afterwards.

## 9.2 Short-Term Detailed Buffer

The detailed buffer is configured as follows:

- control period: 5ms;
- `LOG_SAMPLE_DIVIDER = 4`;
- therefore one record every **20ms**;
- `LOG_BUFFER_SIZE = 120`;
- total history length:

$$120 \times 20ms = 2.4s$$

This buffer stores relatively complete information, including:

- state-machine state;
- line-visible flag;
- boost flag;
- offset, error, derivative, and curve score;
- scheduled `Kp/Kd`, base speed, and max steering authority;
- left and right target speed, actual speed, and duty cycle;
- contrast, line strength, and left-right bias;
- all six raw sensor values.

## 9.3 Long-Term Compressed History

The compressed history buffer is configured as:

- `LONG_LOG_SAMPLE_DIVIDER = 20`;
- therefore one record every **100ms**;
- `LONG_LOG_BUFFER_SIZE = 300`;
- total history length:

$$300 \times 100ms = 30s$$

To reduce RAM usage, this structure stores multiple quantities as scaled `short` or `unsigned short` fields, making it suitable for preserving longer-term behavioral trends.

### 9.4 Freeze and Dump Mechanism

When the system first leaves `STATE_TRACKING` because the line becomes invisible, it:

1. copies the detailed ring buffer into the frozen buffer `frozenBuf`;
2. copies the compressed long-history ring buffer into `frozenLongBuf`;
3. later dumps them in a safe non-real-time context.

The dump policy is:

- in `STATE_LOST_SEARCH`, after a short delay, if no dump has happened yet, the system first prints the long history and then the short detailed history;
- if that does not happen earlier, `STATE_STOPPED_RECOVERY` performs a fallback dump;
- if the frozen detailed buffer is empty, the current live ring is copied and dumped instead.

This mechanism preserves critical evidence leading up to a failure without disturbing real-time control behavior.

## 10 Key Parameter Summary (Current Tuned Parameters)

### 10.1 Control Period and Sensors

- `CONTROL_PERIOD_S = 0.005s` (200Hz)
- `LED_SETTLE_US = 20` $\mu$s
- `ADC_AVG_SAMPLES = 1`

### 10.2 Speed Strategy

- `SPEED_CRUISE = 0.18 m/s`
- `SPEED_STRAIGHT = 0.30 m/s`
- `SPEED_TIGHT = 0.07 m/s`
- `TARGET_SPEED_MAX_TRACK = 0.40 m/s`

## 10.3 Outer Loop

- LINE_KP_STRAIGHT = 0.00010
- LINE_KP_CORNER = 0.00100
- LINE_KD_STRAIGHT = 0.00000
- LINE_KD_CORNER = 0.00003
- CENTER_DEADBAND = 10.0
- DERIVATIVE_LIMIT = 500.0
- MAX_STEER_STRAIGHT = 0.06
- MAX_STEER_CORNER = 0.14

## 10.4 Inner Loop

- WHEEL_KP = 1.2
- WHEEL_KI = 4.0
- WHEEL_FF_GAIN = 2.0
- DUTY_STEP_LIMIT = 0.035
- WHEEL_INT_LIMIT = 0.10

## 10.5 State Machine

- RUN_ARM_VISIBLE_S = 0.25s
- LOST_STOP_TIME_S = 0.025s
- RECOVERY_GO_DELAY_S = 0.20s
- FROZEN_CLEAR_VISIBLE_S = 0.15s

## 10.6 Turnaround Parameters

- TURN_180_TICKS = 960
- TURN_BASE_DUTY = 0.24
- TURN_MIN_DUTY = 0.18
- TURN_SLOW_ZONE_TICKS = 300
- TURN_SYNC_KP = 0.00010

# 11 Conclusion

The main characteristics of the current software architecture are:

- a fixed-rate 200Hz cascade control structure with clear real-time timing;
- ambient-light-compensated sensing for more robust track detection;
- a dynamic PD outer loop combined with curve scoring, speed scheduling, straight-line boost, and steering saturation protection;
- a wheel-speed PI inner loop with feedforward, integrator limiting, and duty slew-rate limiting;
- a state machine supporting power-up arming, loss-stop protection, stable recovery, and BLE-triggered turnaround;
- a dual-level black-box logging system supporting both long history and high-resolution pre-failure reconstruction.

Compared with a simple implementation based only on fixed PID gains and live serial printing, this version is more suitable for high-frequency control, parameter tuning, and abnormal-event analysis on a real physical track.